

RAG-InsurMinds

Aviso Importante

- Os arquivos oficiais deste projeto estão em: <https://github.com/marcelobazevedo/RAG-InsurMinds>
- Para executar, é necessário baixar/clonar esse repositório.
- A execução da aplicação requer ambiente Docker (Docker Engine + Docker Compose).

Aplicação RAG para seguro residencial com:

- interface em Streamlit;
- upload de PDFs;
- ingestão automática para PostgreSQL + pgvector;
- recuperação **dense**, **sparse** e **hybrid**;
- resposta em streaming com fontes citadas.

Documentação complementar:

- [TECNICO.md](#)
- [RELATORIO.md](#)

Vídeo de demonstração

- Link: <https://drive.google.com/file/d/1GF7AtyAyOZ2fKrEI6nbRF6IB2XikERXc/view?usp=sharing>

Pasta de documentos para upload

- Utilize os arquivos que estão na pasta **documentos_para_upload/** para fazer o upload.
- Durante o uso da aplicação, os arquivos enviados ficam em **documentos/** (pasta operacional usada na ingestão e no link de fontes).

Requisitos

- Python **3.12**
- **uv** instalado
- Docker + Docker Compose (execução recomendada)
- Ollama (se **MODELO_LOCAL=true**) **ou** credenciais OpenAI (se **MODELO_LOCAL=false**)

Estrutura rápida

```
RAG-InsurMinds/  
├── app.py  
├── docker-compose.yaml  
├── Dockerfile  
├── .env  
├── .env_sample  
├── documentos_para_upload/      # PDFs de origem para upload manual na  
interface
```

```
|— documentos/                                # PDFs enviados/ingeridos
|— initdb/
|— TECNICO.md
|— RELATORIO.md
|— rag/
|   |— augmented/
|   |— ingest/
|       |— extract_text.py
|       |— truncate_data.py
|       |— reset_project_state.py
|   |— retrieval/
|   |— graph/
```

Configuração de ambiente

1. Copie o arquivo de exemplo:

```
cp .env_sample .env
```

2. Ajuste o `.env`.

Variáveis principais

- `MODELO_LOCAL=true|false`
- `LLM_MODEL` e `EMBEDDING_MODEL` (modo local)
- `OPENAI_API_KEY`, `OPENAI_MODEL`, `OPENAI_EMBEDDING_MODEL` (modo OpenAI)
- `PGVECTOR_DIM` (deve bater com dimensão do embedding)
- `POSTGRES_*` (conexão de banco)
- `CHUNK_SIZE` e `CHUNK_OVERLAP` (ingestão)

Como configurar os modelos

Toda configuração de modelo fica em `.env`.

Opção A: modelo local (Ollama)

```
MODELO_LOCAL=true
LLM_MODEL=ministral-3:14b
EMBEDDING_MODEL=nomic-embed-text:latest
OLLAMA_URL=http://host.docker.internal:11434
```

Neste modo, `OPENAI_API_KEY` pode ficar vazio.

Opção B: OpenAI (ChatGPT API)

```
MODELO_LOCAL=false
OPENAI_API_KEY=cole_sua_chave_aqui
OPENAI_MODEL=gpt-4.1-mini
OPENAI_EMBEDDING_MODEL=text-embedding-3-large
```

A chave deve ser definida em:

```
OPENAI_API_KEY=...
```

Sem ela (com `MODELO_LOCAL=false`), a aplicação não consegue chamar a API da OpenAI.

Uso com Docker (recomendado)

Subir containers

```
docker compose up -d --build
```

Ver status

```
docker compose ps
```

Acessar app

- URL: `http://localhost:8501`

Containers

- `rag_insurminds_postgres`
- `rag_insurminds_streamlit`

Bind mounts usados

- `./documentos:/app/documentos`
- `./documentos:/app/static`

O primeiro é usado pela ingestão. O segundo é usado para abrir os links das fontes no chat.

Uso com uv (host/local)

Instalar dependências

```
uv sync
```

Rodar app local

```
uv run streamlit run app.py
```

Fluxo de upload e ingestão

No app, pela sidebar:

1. Selecione um ou mais PDFs para upload direto.
 - Sugestão: use os arquivos da pasta `documentos_para_upload/`.
2. Regras de validação:
 - somente `.pdf`;
 - até 50 MB por arquivo;
 - sem nome duplicado;
 - até 20 arquivos por envio.
3. O sistema salva os arquivos válidos em `documentos/` (pasta operacional).
4. A ingestão automática é executada em sequência (arquivo a arquivo) com barra de progresso.
5. Ao final, o app mostra resumo de:
 - arquivos salvos;
 - arquivos rejeitados (com motivo);
 - ingestões concluídas e falhas.
6. Chunks e embeddings são gravados na tabela `dados`.

Ingestão manual (opcional)

Dentro do container

```
docker compose exec streamlit uv run --no-sync python -m  
rag.ingest.extract_text
```

No host

```
uv run python -m rag.ingest.extract_text
```

Limpeza de arquivos e base

No app existe o botão **Remover todos os arquivos**, que:

- remove os PDFs de `documentos/`;
- executa `TRUNCATE TABLE dados RESTART IDENTITY`.

Comandos úteis

Logs

```
docker compose logs -f streamlit
docker compose logs -f postgres
```

Reiniciar containers

```
docker compose down
docker compose up -d --build
```

Recriar sem rebuild

```
docker compose up -d --force-recreate
```

Troubleshooting

`connection refused` no PostgreSQL

- Verifique containers: `docker compose ps`
- Se rodar no host, ajuste `.env` para porta publicada:
 - `POSTGRES_HOST=localhost`
 - `POSTGRES_PORT=54321`

App sobe, mas não responde bem

- Confirme modelos no `.env`.
- Em modo local, valide `OLLAMA_URL`.

Upload não aparece na pasta local

- Verifique mount:
 - `./documentos:/app/documentos`

Link da fonte não abre

- Confirme se o arquivo existe em `documentos/`.
- Confirme mount estático:
 - `./documentos:/app/static`
- Padrão de link esperado no app:

- `/app/static/<nome_do_arquivo>.pdf`